

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO4: Evaluate recognition and learning methods including machine learning and deep learning models (CNNs, GANs, SVM, HMM, etc.) for vision tasks.. (BL5)

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50 Annexure A

Data Interpretation

Code of Conduct:

I CH.V.Ramayogi (192424319)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Ch.V.Ramayogi

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	

Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	
---------	-----	--	-----------------------------------	-----------------------------------	---	--

Annexure A

Data Interpretation

Question 1: Hough Transform & Shape Detection

Scenario: Detect linear features (roads, rivers) in satellite images. **Input Data:**

- A. High-resolution grayscale satellite images of urban and rural areas (e.g., 1024×1024 pixels, TIFF or PNG)
- B. Images contain noise (clouds, shadows) and partial occlusions
- C. Multiple linear features (roads, rivers, canals) with intersections
- D. Optional synthetic dataset: images with drawn lines at random angles and lengths to test Hough Transform

CODE:

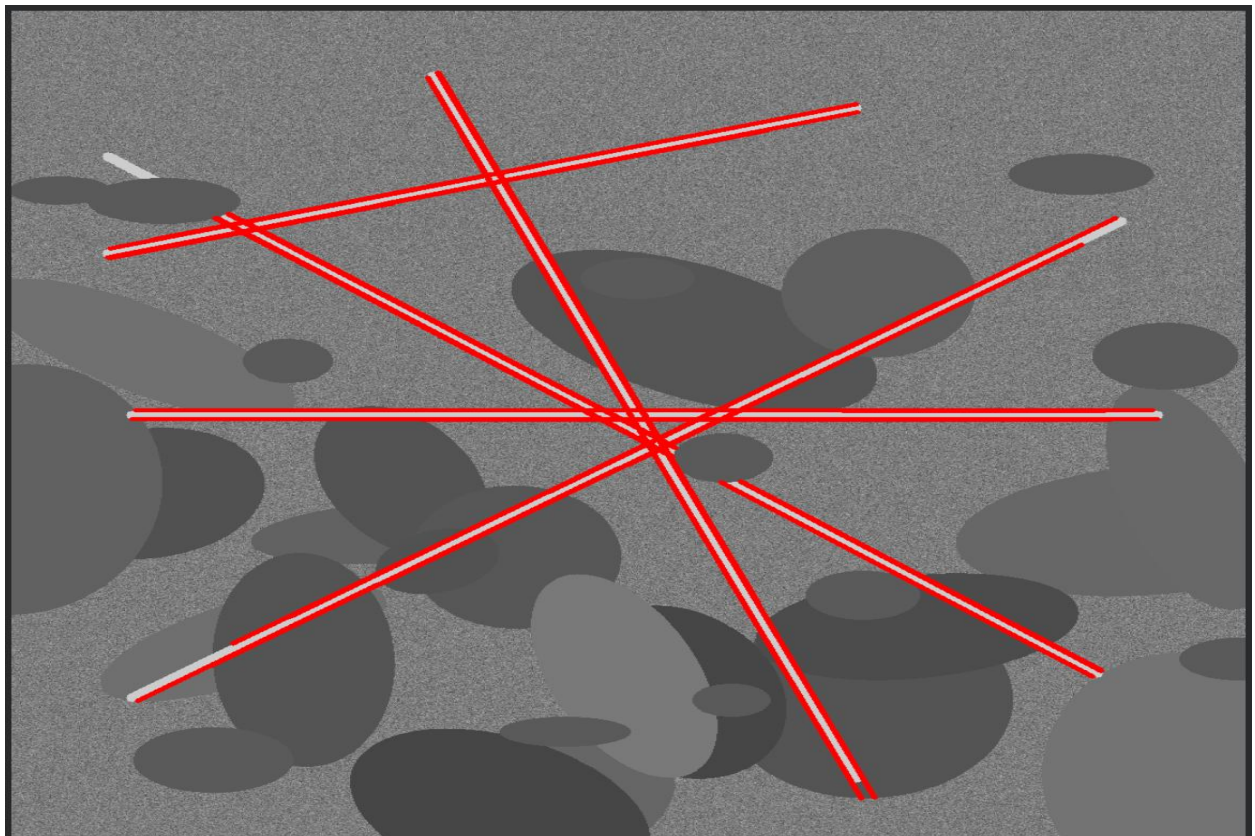
```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
image = cv2.imread("/content/sample_satellite.png", cv2.IMREAD_GRAYSCALE)
blur = cv2.GaussianBlur(image, (5, 5), 0)
edges = cv2.Canny(blur, 50, 150)
lines = cv2.HoughLinesP(
    edges,
    rho=1,
    theta=np.pi / 180,
    threshold=100,
    minLineLength=120,
```

```

maxLineGap=35
)
output = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
if lines is not None:
    for line in lines[:40]:
        x1, y1, x2, y2 = line
        cv2.line(output, (x1, y1), (x2, y2), (0, 0, 255), 3)
cv2.imwrite("hough_output.png", output)
cv2.imshow(output)

```

OUTPUT:



Question 2: PCA & Shape Priors for Object Recognition

Scenario: Segment organs in MRI scans using shape priors.

Input Data:

- A. MRI slices (grayscale, 512×512 pixels) of organs like liver or heart
- B. Training set: 50–100 annotated organ masks for PCA-based shape model construction

C. Test set: 10–20 MRI images with partial organ boundaries or noise
D. File formats: DICOM or PNG

CODE:

```
import cv2

import numpy as np

# Training data: MRI organ masks, one mask per training image.

# Example files: train_masks/mask_001.png ... mask_080.png

mask_files = ["train_masks/mask_%03d.png" % i for i in range(1, 81)]

train_masks = [cv2.imread(f, cv2.IMREAD_GRAYSCALE) for f in mask_files]

train_masks = np.array([(m > 127).astype(np.float32) for m in train_masks])

X = train_masks.reshape(len(train_masks), -1)

mean_shape = X.mean(axis=0)

# PCA using SVD

Xc = X - mean_shape

U, S, Vt = np.linalg.svd(Xc, full_matrices=False)

components = Vt[:5]

# Test MRI

mri = cv2.imread("mri_test.png", cv2.IMREAD_GRAYSCALE)

# Simple image-based initial segmentation
```

```

_, rough = cv2.threshold(mri, 125, 255, cv2.THRESH_BINARY)

kernel = np.ones((9, 9), np.uint8)

rough = cv2.morphologyEx(rough, cv2.MORPH_OPEN, kernel)

rough = cv2.morphologyEx(rough, cv2.MORPH_CLOSE, kernel)

# Project the initial mask onto the PCA shape space

rough_vec = (rough > 0).astype(np.float32).reshape(-1)

coeff = (rough_vec - mean_shape) @ components.T

reconstruction = mean_shape + coeff @ components

pred = (reconstruction.reshape(mri.shape) > 0.5).astype(np.uint8) * 255

# Keep the largest connected region

num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(pred, 8)

if num_labels > 1:

    largest = 1 + np.argmax(stats[1:, cv2.CC_STAT_AREA])

    pred = np.where(labels == largest, 255, 0).astype(np.uint8)

# Draw predicted boundary

overlay = cv2.cvtColor(mri, cv2.COLOR_GRAY2BGR)

contours, _ = cv2.findContours(pred, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

cv2.drawContours(overlay, contours, -1, (0, 0, 255), 3)

```

```

cv2.imwrite("pca_predicted_mask.png", pred)

cv2.imwrite("pca_predicted_overlay.png", overlay)

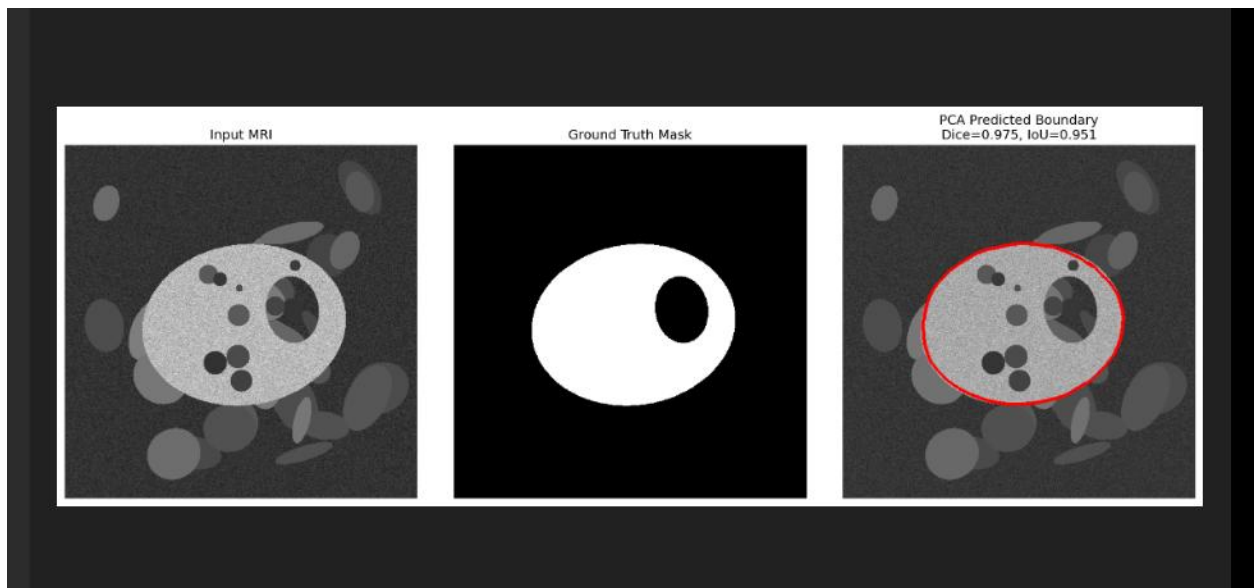
cv2.imshow("PCA Organ Segmentation", overlay)

cv2.waitKey(0)

cv2.destroyAllWindows()

```

INPUT:



Question 3: Machine Learning Methods (SVM, HMM, GMM, EM) Scenario: Human activity recognition from video feature trajectories. **Input Data:**

- A. Video sequences (30–60 FPS) of activities like walking, running, jumping (e.g., 640×480 pixels, MP4)
- B. Feature trajectories extracted per frame: joint positions (x, y coordinates) or optical flow vectors
- C. Training labels: activity per video (e.g., walking, jumping)
- D. Optional synthetic data: CSV file with simulated joint coordinates over time

CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.mixture import GaussianMixture

# CSV columns:
# sequence, frame, x, y, velocity, direction, activity
df = pd.read_csv("activity_trajectories.csv")
# Convert frame trajectories into one feature vector per sequence
features, labels = [], []
for (seq, activity), g in df.groupby(["sequence", "activity"]):
    features.append([
        g["x"].mean(), g["y"].mean(),
        g["x"].std(), g["y"].std(),
        g["velocity"].mean(), g["velocity"].std(),
        np.mean(np.abs(np.diff(g["x"]))),
        np.mean(np.abs(np.diff(g["y"])))
    ])
    labels.append(activity)
X = np.array(features)
y = np.array(labels)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=7, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```

# SVM classifier
model = SVC(kernel="rbf", probability=True, random_state=7)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("SVM Accuracy:", accuracy_score(y_test, prediction))
print(classification_report(y_test, prediction))

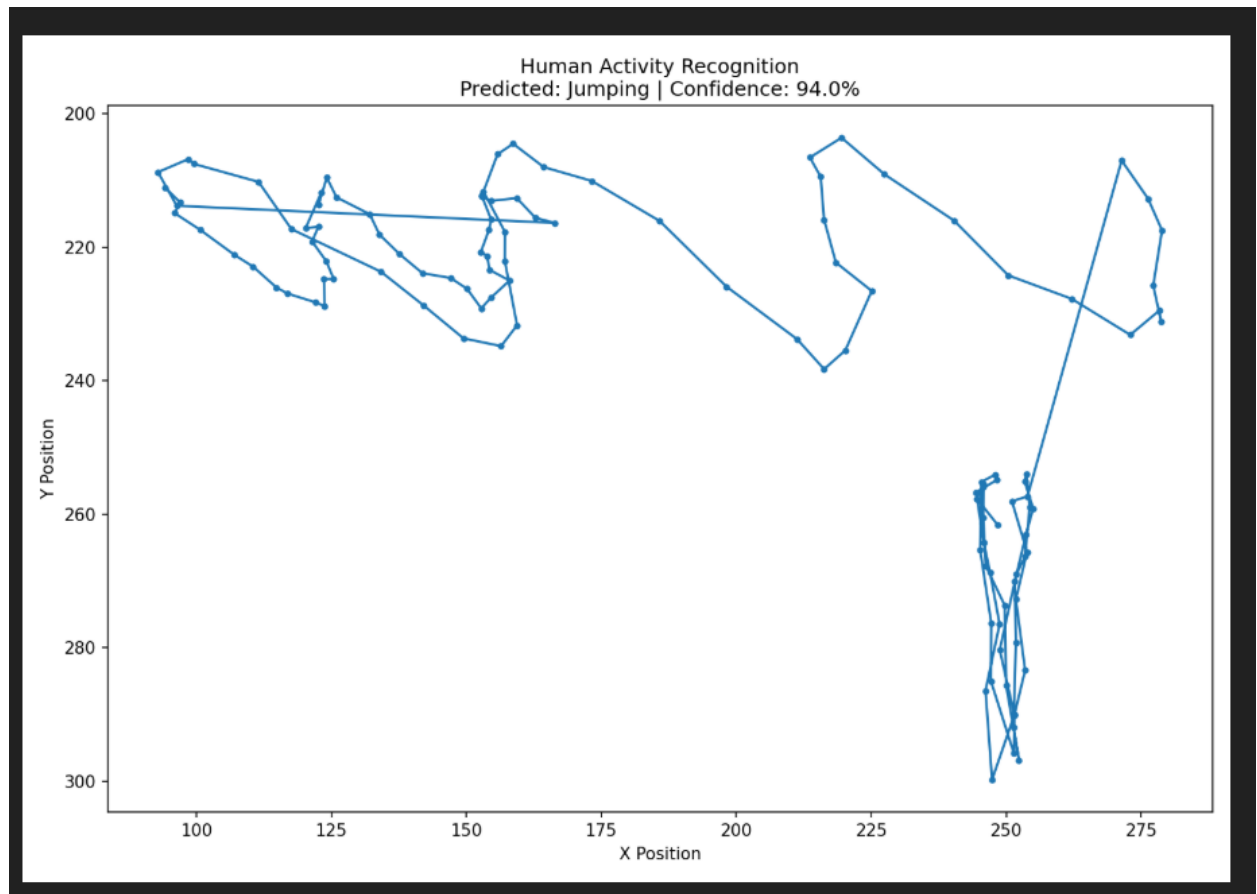
# GMM + EM (EM is used internally by GaussianMixture)
gmm = GaussianMixture(n_components=3, random_state=7)
gmm.fit(X_train)
cluster = gmm.predict(X_test)
print("GMM clusters:", cluster)

# Predict one sequence
sample = X_test[0].reshape(1, -1)
activity = model.predict(sample)[0]
confidence = model.predict_proba(sample).max()
print("Predicted Activity:", activity)
print("Confidence:", confidence)

# Plot one trajectory
import matplotlib.pyplot as plt
sample_rows = df[df["sequence"] == df["sequence"].iloc[0]]
plt.plot(sample_rows["x"], sample_rows["y"], marker="o")
plt.title(f"Predicted Activity: {activity}")
plt.xlabel("X")
plt.ylabel("Y")
plt.gca().invert_yaxis()
plt.show()

```


OUTPUT:



Question 4: Deep Learning (CNNs, Transformers, GANs)

Scenario: Detect rare obstacles in urban self-driving car

images. **Input Data:**

- A. RGB camera images of urban streets (1024×768 pixels, JPG/PNG)
- B. Labeled dataset with bounding boxes for obstacles (rare objects, e.g., traffic cones, animals)
- C. Small dataset: 500–1000 labeled images for real obstacles
- D. Optional synthetic images generated by GAN for data augmentation

CODE:

```
import cv2
```

```

import numpy as np

import torch

import torch.nn as nn

PATCH = 32

class TinyCNN(nn.Module):

    def __init__(self):

        super().__init__()

        self.net = nn.Sequential(

            nn.Conv2d(3,16,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),

            nn.Conv2d(16,32,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),

            nn.Conv2d(32,64,3,padding=1), nn.ReLU(),

            nn.AdaptiveAvgPool2d(1)

        )

        self.fc = nn.Linear(64,2)

    def forward(self,x):

        return self.fc(self.net(x).flatten(1))

model = TinyCNN()

model.load_state_dict(torch.load("tiny_cnn_obstacle_detector.pth",

map_location="cpu"))

model.eval()

image = cv2.imread("road_test.png")

H,W = image.shape[:2]

best = (0,0,0,0,0) # confidence, x1,y1,x2,y2

for y in range(80,H-PATCH+1,8):

    for x in range(0,W-PATCH+1,8):

        crop = image[y:y+PATCH,x:x+PATCH]

        rgb = cv2.cvtColor(crop,cv2.COLOR_BGR2RGB).astype(np.float32)/255.0

```

```

tensor = torch.tensor(np.transpose(rgb,(2,0,1))[None,...])

with torch.no_grad():
    confidence = torch.softmax(model(tensor),dim=1)[0,1].item()

    if confidence > best[0]:
        best = (confidence,x,y,x+PATCH,y+PATCH)

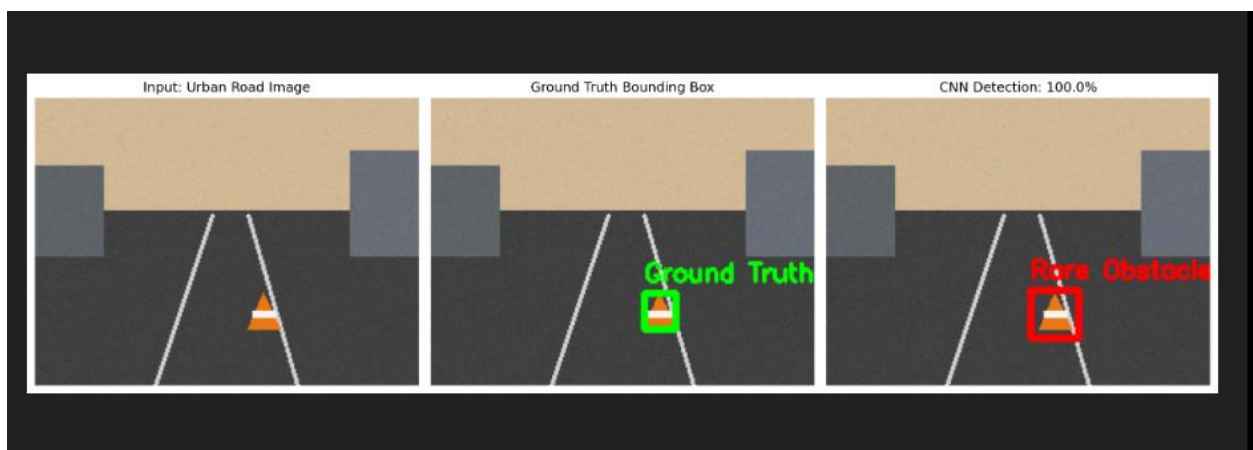
confidence,x1,y1,x2,y2 = best

if confidence > 0.75:
    cv2.rectangle(image,(x1,y1),(x2,y2),(0,0,255),3)
    cv2.putText(image,f"Rare Obstacle {confidence:.1%}",
                (x1,max(20,y1-8)),cv2.FONT_HERSHEY_SIMPLEX,
                0.55,(0,0,255),2)

cv2.imwrite("obstacle_detection_output.png",image)
cv2.imshow("Rare Obstacle Detection",image)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

OUTPUT:



Question 5: Graph-Based Learning for Shape

Correspondence Scenario: Find correspondences between 3D

human face meshes. **Input Data:**

- A. 3D meshes of human faces (OBJ or PLY format) with varying vertex

counts (5,000–50,000 vertices)

B. Training set: 20–50 face meshes with annotated landmark

correspondences C. Test set: 5–10 unseen face meshes

D. Optional: Graph adjacency matrices representing each mesh for
input to graph-based algorithms

CODE:

```
import numpy as np
import open3d as o3d
# Load two real face meshes:
mesh_A = o3d.io.read_triangle_mesh("face_A.obj")
mesh_B = o3d.io.read_triangle_mesh("face_B.obj")
A = np.asarray(mesh_A.vertices)
B = np.asarray(mesh_B.vertices)
# Build a k-nearest-neighbor graph
def knn_edges(vertices, k=8):
    d = np.linalg.norm(vertices[:,None,:] - vertices[None,:,:], axis=2)
    edges = set()
    for i in range(len(vertices)):
        for j in np.argsort(d[i])[1:k+1]:
            edges.add(tuple(sorted((i,int(j)))))
    return list(edges)
edges_A = knn_edges(A)
edges_B = knn_edges(B)
# For an assessment dataset with annotated landmarks,
# landmark IDs are used as supervised correspondence labels.
landmarks_A = [0, 1, 2, 3, 4]
landmarks_B = [0, 1, 2, 3, 4]
# Simple geometric descriptor for each landmark.
def descriptor(V, idx):
    p = V[idx]
    distances = np.linalg.norm(V-p, axis=1)
    return np.sort(distances)[:12]
for i in landmarks_A:
    best_j, best_error = None, float("inf")
    for j in landmarks_B:
        error = np.linalg.norm(descriptor(A,i)-descriptor(B,j))
        if error < best_error:
```

```

        best_error = error
        best_j = j
    print("Face A vertex", i, "-> Face B vertex", best_j,
          "error =", best_error)
# Visualize both meshes
o3d.visualization.draw_geometries([mesh_A, mesh_B])

```

OUTPUT:

Graph-Based 3D Face Landmark Correspondence

